Cloud Computing · Batch 18

Nightly Batch Jobs with Deadlines

Greedy + Disjoint Set Union for maximum-profit unit-job scheduling

8-job worked example

Real load: 10^6 jobs, 10^4 slots

Must finish in under 2 s

Technique: Greedy + DSU Rejected (implemented): Backtracking

24I302 – Abirami P C
24I308 – Bharaghave Shi C G
24I314 – Dhekshika D
24I358 – Sowmya P Nagarajan
24I374 – Vinothini

What we need to schedule

Input

n jobs. Job j has a deadline d_j (in hours, 1 to D) and a profit $p_j > 0$. Real load: $n = 10^6$, $D = 10^4$.

Output

A set S of jobs and a slot s(j) between 1 and d_j for each job in S, with all slots distinct.

Goal

Maximise the total profit of the jobs in S. A late job pays nothing.

Constraints

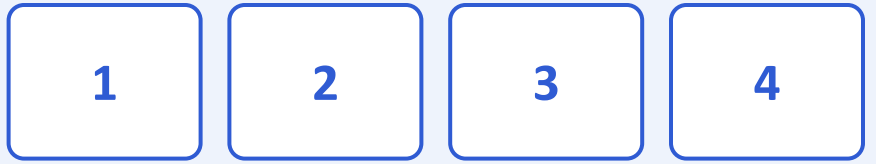
- One job per slot
- Unit duration, no preemption
- Finish by the deadline: $s(j) \le d_j$
- Whole schedule in under 2 s at $n = 10^6$

Problem type: job sequencing with deadlines. The feasible job sets form a matroid, so greedy is exact.

A small example to work through

Job	Deadline (hour)	Profit (Rs thousand)
J1	2	100
J2	1	19
J3	2	27
J4	1	25
J5	3	15
J6	4	60
J7	3	40
J8	4	30

Available slots



Slots 1 to 4 (the largest deadline is 4), so at most 4 jobs can run.

The question

Which jobs go into which slots to earn the most profit?

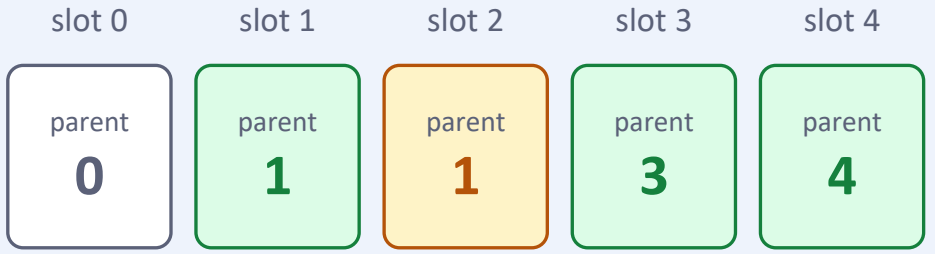
Finding the latest free slot quickly

1. Keep parent[s] for every slot 0 to D. A free slot points to itself.

2. find(d) follows the parents and returns the latest free slot at or before d. Result 0 means none.

3. When slot s is taken, set parent[s] = find(s − 1), so later searches skip it.

After J1 takes slot 2



Slot 2 is taken, so it now points to slot 1.

find(2) lands on slot 1 in one hop.

Green = free, amber = taken. Slot 0 means no free slot.

Four ways to find it, and only some are fast enough

Structure	Cost per find	Total work at n = 10^6	Measured (random / worst)	Meets 2 s?
Linear scan	O(D)	up to 10^{10} probes	0.74 s / 2.18 s	**No (worst case)**
Balanced BST (TreeSet)	O(log D)	≈ 1.3×10^7	15 ms / 5.5 ms	**Yes**
Segment tree	O(log D)	≈ 2.7×10^7	3.9 ms / 2.1 ms	**Yes**
DSU + path compression	≈ O(α(D))	≈ 10^6 hops	2.6 ms / 1.6 ms	**Yes**

Why DSU wins: after path compression a find follows about one pointer. The sort (76 ms for 2.0×10^7 comparisons) becomes the only O(n log n) cost.

End to end, Greedy + DSU takes 93 ms for 10^6 jobs.

The greedy rule

1. **Sort jobs by profit, highest first**
2. **Place each job in the latest free slot at or before its deadline**
3. **No free slot? Reject the job**

Greedy-choice property

The most profitable job that still fits belongs to some optimal solution.

Optimal substructure

After fixing that job, what remains is the same problem on the other jobs.

Why it works

A set is feasible when, for every t, at most t jobs have deadline ≤ t. Feasible sets form a matroid, so greedy by profit is optimal.

Why the other four techniques fail

Technique	Why it is unsuitable	Concrete number
Brute force	Tries every subset (2^n) or ordering ($n!$) and tests feasibility.	$n = 30$ gives $2^{30} \approx 10^9$ subsets. $n = 10^6$ is hopeless.
Divide & conquer	The halves compete for the same slots, so merging means re-solving.	Split the 8 jobs by deadline: 127 + 100 = 227, below the optimum 230.
Dynamic programming	Correct, but the table over (job, slots) costs $O(nD)$.	10^{10} updates (about 6 s) against 2×10^7 for greedy + DSU.
Backtracking (implemented)	Include/exclude search. Pruning alone never makes it polynomial.	$n = 30$: $2^{31} - 1$ nodes, 6.3 s. $n = 40$ would take about 1.8 h.

Backtracking finds the same optimum as greedy on all 10 test inputs. Branch-and-bound is pruned backtracking and keeps the exponential worst case.

The algorithm

```
SCHEDULE(jobs[1..n], D):
  sort jobs by profit, descending
  for s = 0..D: parent[s] = s
  total = 0
  for each job j in sorted order:
    s = FIND(min(d_j, D))
    if s > 0:
      slot[s] = j;  total += p_j
      parent[s] = FIND(s - 1)
    // else: reject j
  return total, slot

FIND(x):   // with path compression
  r = x
  while parent[r] != r: r = parent[r]
  while parent[x] != r:
    nxt = parent[x]
    parent[x] = r;  x = nxt
  return r
```

State

parent[0..D] over the slots. Slot 0 is a sentinel meaning no free slot.

Invariant

FIND(x) is the largest free slot ≤ x.

Greedy rule

Highest profit first, latest free slot, reject if none.

Tracing the 8 jobs

Step	Job (d, p)	find(d)	Decision	DSU update	parent[0..4]
1	J1 (2, 100)	2	**slot 2**	parent[2] = find(1) = 1	[0, 1, 1, 3, 4]
2	J6 (4, 60)	4	**slot 4**	parent[4] = find(3) = 3	[0, 1, 1, 3, 3]
3	J7 (3, 40)	3	**slot 3**	parent[3] = find(2) = 1	[0, 1, 1, 1, 3]
4	J8 (4, 30)	4 → 3 → 1 = 1	**slot 1**	compress 4, 3 → 1; parent[1] = 0	[0, 0, 1, 1, 1]
5	J3 (2, 27)	2 → 1 → 0 = 0	**reject**	compress: parent[2] = 0	[0, 0, 0, 1, 1]
6	J4 (1, 25)	0	**reject**	none	[0, 0, 0, 1, 1]
7	J2 (1, 19)	0	**reject**	none	[0, 0, 0, 1, 1]
8	J5 (3, 15)	3 → 1 → 0 = 0	**reject**	compress: parent[3] = 0	[0, 0, 0, 0, 1]

J8 takes slot 1 because slots 4, 3 and 2 are taken: DSU jumps straight to it. After step 4 all four slots are full, so every later job is rejected.

The best schedule earns Rs 230 thousand

J8	J1	J7	J6
Rs 30k	Rs 100k	Rs 40k	Rs 60k
hour 0–1	hour 1–2	hour 2–3	hour 3–4

Total profit

30 + 100 + 40 + 60 = **Rs 230 thousand**

Rejected: J3, J4, J2, J5 (27 + 25 + 19 + 15 = 86)

Optimality check

Only 4 slots exist, so at most 4 jobs run. The four largest profits (100, 60, 40, 30) fit together, so 230 is an upper bound and it is reached.

Earliest-deadline-first loses profit

Greedy + DSU

230

Rs thousand, optimal

J8, J1, J7, J6

EDF, best tie-break

225

Rs thousand, loses 5

J4, J1, J7, J6

EDF, ties by input order

194

Rs thousand, loses 36

J2, J1, J5, J6

EDF never looks at profit. J2 (19) blocks J4 (25) at deadline 1, J5 (15) blocks J7 (40) at deadline 3, and J8 never gets a slot.

EDF is the right rule to meet every deadline when that is possible. It is not the right rule to maximise profit when only some jobs fit.

Why greedy is optimal, part 1: the feasibility test

Lemma 1 **Feasibility**

A set A can be scheduled exactly when, for every t, at most t jobs in A have deadline ≤ t.

(⇒) Those jobs need distinct slots among 1..t.

(⇐) Sort A by deadline and put the k-th job in slot k. The count condition gives $k \le d_k$.

Lemma 2 **The DSU test is exact**

With latest-free-slot placement, find(d_j) = 0 exactly when adding j makes the set infeasible.

If find(d_j) = 0, slots 1..d_j are full. Let t* be the largest value with slots 1..t* full.

A job in those slots with deadline > t* would have been placed after t*. So t* jobs have deadline ≤ t*, and with j that makes t* + 1. Infeasible by Lemma 1.

Why greedy is optimal, part 2: the exchange argument

Theorem. Greedy (profit descending, accept if feasible) returns a maximum-profit feasible set G.

Take an optimal set O that agrees with greedy on the longest prefix of decisions. Suppose they first differ at job j_i, and let P be the jobs both have accepted before it.

Case 1 **O contains j_i, greedy rejected it**

$P \cup \{j_i\}$ is a subset of O, so it is feasible. But greedy found it infeasible. Contradiction.

Case 2 **Greedy accepted j_i, O does not contain it**

Some later job x in O is no more profitable than j_i. Swap it: $O' = O - x + j_i$ is feasible with profit at least that of O, and agrees with greedy on a longer prefix. Contradiction.

Both cases fail, so O = G and greedy is optimal.

Complexity: O(n log n), dominated by the sort

Where the time goes

- Sort by profit: $\Theta(n \log n)$, about 2.0×10^7 comparisons at $n = 10^6$
- Initialise DSU: $O(D)$
- n finds and at most D unions: $O((n + D)\,\alpha(D))$, with $\alpha(D) \leq 4$
- **Total: O(n log n)**. Space: $O(n + D)$

Best

$O(n + D)$ with counting sort on bounded integer profits.

Worst

Sort-bound $O(n \log n)$. A chain input makes one find walk D nodes, and compression pays for it once.

Practical

10^6 jobs in about 93 ms: 76 ms sort, about 2 ms DSU, 1.0 parent hop per job.

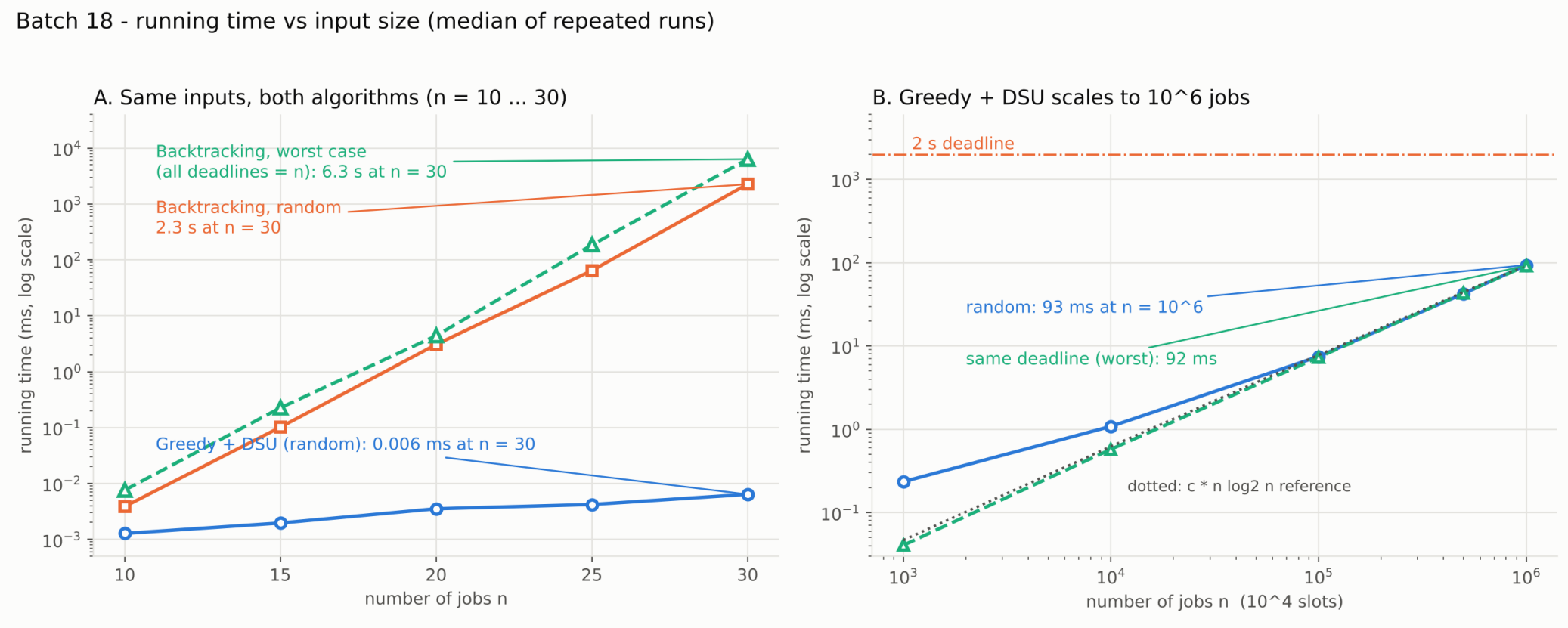
For comparison: Backtracking $O(2^n D)$ · linear-scan variant $O(nD)$ · DP $O(nD)$

Worst-case inputs we tested

Workload	How it is built	What happens ($n = 10^6$, $D = 10^4$)
Chain (DSU stress)	The highest-profit jobs have deadlines D, D − 1, …, 1. All other jobs have deadline D.	One find walks 10^4 parents, then compression makes later finds cost 1 hop. About 10^6 hops in total, find phase 2.7 ms.
Same deadline (maximum contention)	Every job has deadline D.	The first D jobs fill the slots and 990,000 are rejected. DSU: 1.6 ms. Linear scan: 9.95×10^9 probes, 2.18 s, over the limit.
Same deadline, D = n (backtracking killer)	All deadlines equal n, so every subset is feasible and nothing is pruned.	The search tree has exactly $2^{n+1} - 1$ nodes: n = 20 takes 6 ms, n = 25 takes 186 ms, n = 30 takes 6.3 s.

Greedy + DSU stays at 92–104 ms on every workload at 10^6 jobs. The sort dominates, as the analysis predicts.

Timing results



Setup

Java 21, 2 cores, median of 5–9 runs (1 for the slowest backtracking cases), fixed seeds. Data in timing.csv (40 rows, 3 workloads).

Both algorithms, n = 10 to 30

Greedy under 0.01 ms. Backtracking 0.004 ms up to 2.3 s (random) and 6.3 s (worst). Profits identical in all 10 comparisons.

Greedy only, n = 10^3 to 10^6

0.07 ms up to 93 ms, growing like n log n, about 20× below the 2 s limit.

How this relates to real systems

Algorithm reference

Gabow & Tarjan, A linear-time algorithm for a special case of disjoint set union, J. Computer and System Sciences 30(2):209–221, 1985. It covers union-find on a line, our pattern of merging a taken slot into the slot below.

Real system

The Linux kernel SCHED_DEADLINE class (earliest-deadline-first with a constant bandwidth server), documented in Documentation/scheduler/sched-deadline.

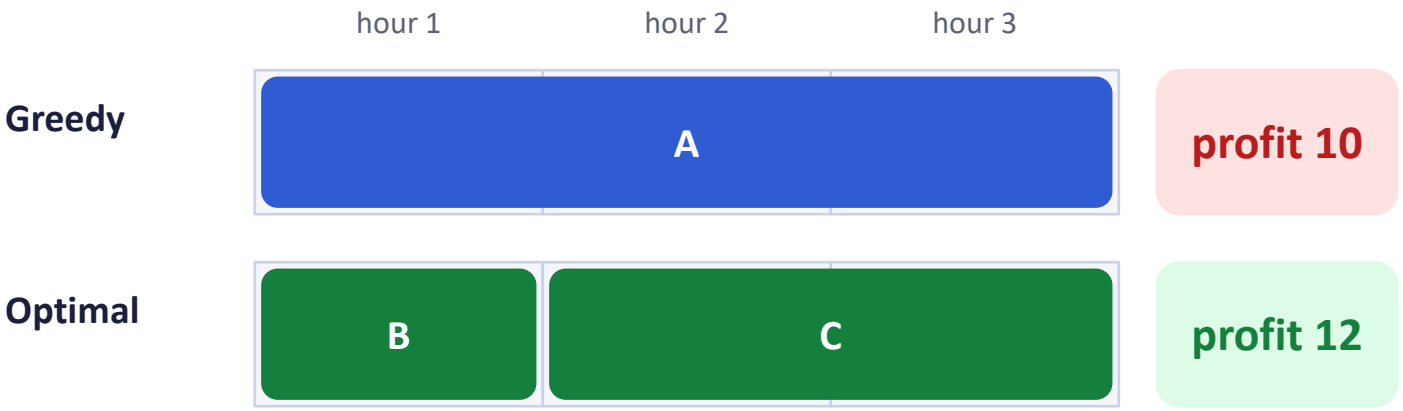
How this project differs

	Our batch scheduler	SCHED_DEADLINE
Goal	Maximise profit of on-time jobs	Meet all admitted deadlines
Rule	Profit-greedy, latest free slot	EDF on absolute deadlines, preemptive
Jobs	One-off unit jobs, offline	Periodic or sporadic tasks with runtime budgets
Overload	Reject low-profit jobs	Admission control, budget enforcement

What if jobs last 1 to 4 hours?

Jobs (duration, deadline, profit) in a 3-hour window

A = (3 h, d = 3, 10) B = (1 h, d = 1, 6) C = (2 h, d = 3, 6)



Greedy takes A first. It fills hours 1 to 3, so B and C are both blocked.

Why greedy breaks

The greedy-choice property is lost: one high-profit job can displace two lower ones.

Feasible sets are no longer a matroid.

One DSU slot per job no longer models the problem.

VariableDurationDP.java prints 10 versus 12.

New technique: dynamic programming

Lawler–Moore style DP

Sort jobs by deadline. Let f[t] be the best profit of a chosen subset with total length exactly t.

For each job (p, d, w), with duration p and profit w, go through $t \le d - p$ downward:

```
f[t + p] = max(f[t + p],  f[t] + w)
```

How to meet 2 s

- Split the work over cores or use bitset tricks
- Prune dominated jobs
- Use a profit-scaling approximation scheme

O(nD)

time, with O(D) space

10^{10}

updates at $n = 10^6$, $D = 10^4$

≈ 6.5 s

at 10^6 jobs (measured 645 ms at $n = 10^5$), above the 2 s target

General durations make the problem NP-hard (it contains knapsack), so a pseudo-polynomial DP is the standard exact tool.

Summary

Q1 J8, J1, J7, J6 in hours 1–4, profit 230

Q2 Exchange argument over a matroid (slides 12–13)

Q3 EDF gives 194 (225 with the best tie-break), below 230

Q4 DSU ≈ 10^6 hops vs 10^{10} probes (linear), 1.3×10^7 (BST), 2.7×10^7 (segment tree)

Q5 3-job example gives 10 vs 12, so use DP, O(nD)

Takeaways

- Greedy + DSU: O(n log n), 93 ms for 10^6 jobs, optimal.
- Backtracking: same answer, but 2^n. Unusable beyond n ≈ 35.
- Linear scan is correct but hits the 2 s wall on the worst input.
- Changing a requirement (durations) changes the technique, not just the constants.

Deliverables: slides, analysis sheet, GreedyDSU.java, BacktrackingScheduler.java, README, timing.csv, graph, video, GitHub repo.